Source: docs\WEBSITE_CONTENT.md | Export: 11 November 2026

Website text

The Content Overview at `/admin/website-content` and fixed editors at `/admin/website-content/{section}` manage text in the existing `siteSettings/storeContent` document. This is not a page builder. Contact Information, About Us, Banners, Campaigns and Coupons keep their existing sections.

Registry and public mapping

The shared registry is `src/content/sections.ts`; it is also the server allowlist. Unsupported identifiers, arbitrary paths and unexpected fields fail validation.

Section	Stored fields	Public renderer / route
`home-featured`	`textSections.home-featured.{heading,supportingText,description}`	Existing HomeView collection grid, `/`
`home-categories`	`textSections.home-categories.{heading,supportingText,description}`	Existing featured-category navigation, `/`, when configured categories exist
`shipping`	`policies.shipping.{heading,text,approved}`	PoliciesView, `/shipping-policy`
`returns`	`policies.returns.{heading,text,approved}`	PoliciesView, `/returns-policy`
`privacy`	`policies.privacy.{heading,text,approved}`	PoliciesView, `/privacy-policy`
`terms`	`policies.terms.{heading,text,approved}`	PoliciesView, `/terms-conditions`
`footer-brand`	`textSections.footer-brand.text`	Footer brand column, beside the unchanged contact information
`footer-support`	`textSections.footer-support.text`	Existing Footer location/support paragraph

The existing `/policies?tab=…` links also remain compatible. Changing a heading does not change the canonical URL or policy navigation label. Missing policy headings retain their canonical title. Missing homepage/footer values use only the pre-existing static compatibility behavior; the overview counts saved text, not those fallbacks. Explicitly clearing homepage/footer copy stores empty text and marks the entry Not Set. Policy text must be nonempty and owner-approved before saving.

There is no standalone New Arrivals block in the current homepage. The implementation exposes the two actual homepage areas instead of creating another product grid or altering product assignments. Eight supported entries therefore replace the export's inconsistent eight/nine counts. The footer supporting text stays in its actual existing paragraph rather than relocating the footer. Prototype policy examples, legal commitments, certifications, audit percentages, Redis claims and fictional identities/timestamps are not copied.

API, isolation and revisions

`website:list`, `website:get` and `website:save` reuse `/api/catalog`. Protected reads and writes require BOTH `admin: true` and enabled `adminAccess/{uid}`. Writes recheck the enabled record in the transaction. The client cannot supply document paths or author/timestamp metadata.

Reads do not initialize or seed documents. The first explicit successful save can create the normal content document. Updates use an explicit per-field allowlist and share the existing document revision with banners/contact settings. A selected policy save touches only that policy's heading/text/approval plus revision and metadata; homepage/footer saves touch only their fixed text field. Unrelated settings and unknown fields remain intact.

`textSectionMeta.{section}` stores a server timestamp, actor, operation ID and content hash. The administrator API exposes the timestamp and whether it is section-specific or a shared-document fallback, not the actor or operation. Identical same-actor retry operations are idempotent. Stale revisions and changed operations fail instead of overwriting another administrator's edits. The conflict UI keeps entered text, loads the latest saved text for comparison and requires an explicit choice before rebasing or replacing the form.

Text edits never write shipping calculations, products, assignments, orders, inventory, refunds or `siteSettings/returns`. Existing operational return configuration and its approved snapshot remain independent. The legacy shared-content save skips rewriting that operational snapshot when the submitted return configuration/policy is unchanged, so a subsequent contact save does not silently resynchronize a new wording-only edit. Actual changes through that legacy configuration path retain their existing validation.

Existing policy approval is preserved. A policy editor requires the owner's approval confirmation when absent; Content Added only means saved text exists and never claims legal verification. Legacy unapproved policy text/headings stay private. No Draft/Publish, autosave, scheduling or history UI is introduced.

Rendering, preview and refresh

`TextCopy.tsx` supplies the exact homepage heading/intro/description, footer text and policy heading/body components used by the actual public pages and private preview. Text is rendered through React string children with preserved paragraph whitespace; HTML is never interpreted. There is no public preview API or unauthenticated preview flag. Previews use current unsaved values and explicitly say **Preview - unsaved changes.** They do not save or publish anything.

Lengths: headings 100, homepage supporting text 500, homepage description/footer text 2,000, policy body 12,000 JavaScript string code units. Control characters are rejected; Unicode and internal paragraphs are retained, CRLF is normalized and surrounding whitespace is trimmed. Static fallback content is not loaded into the form as if it were saved.

The existing no-store public content service returns only allowlisted text and eligible banners; it omits private settings and admin metadata. Homepage, footer and policies share its refresh behavior: revisit/refresh fetches immediately; open pages update within 15 seconds or at an earlier banner boundary. Public content fields are projected from one document snapshot for consistency. No CDN/cache or scheduler setup is required.

The editor loads once per section and aborts stale requests when leaving. Focus/visibility changes do not replace edited text. Mobile Edit/Preview tabs preserve the form; the fixed Save/Cancel area reserves layout space and honors safe areas. Native discard confirmations protect actual navigation and Cancel. Preview device controls do not change persistence.

Checks and release

Focused tests are `src/content/sections.test.tsx`, `scripts/website-content-emulator.test.ts` and `scripts/website-content-browser.test.py`, with loopback-only fixture tooling. The existing admin test harness includes the new emulator/browser checks. They cover fixed paths, initial reads without writes, Unicode/HTML safety, all section saves/public rendering, per-section versus document timestamps, shared-field preservation, operational return isolation, conflicts, retry idempotency, private-preview isolation, mobile tabs/Save actions and desktop/mobile widths. Banner/business regressions and packaged function-runtime checks remain applicable.

No new dependencies, serverless functions, Firestore indexes, rule publication or production migration are required. The existing rules deny direct access to the shared private settings document. Local/emulator/build evidence does not establish hosted acceptance. The owner must use the normal commit/push/application deployment process and then check saved copy, public refresh and existing authorized access on the hosted site.

Implementation and tests modify local source and emulator fixtures only. No production content, users, business records, media, rules or indexes are changed.

The responsive customer policy layout, canonical route mapping, explicit local sample preview and production missing-content behavior are documented in POLICY_PAGE_DESIGN.md. Real policy bodies remain plain text; their presentation does not infer section headings or change approval.